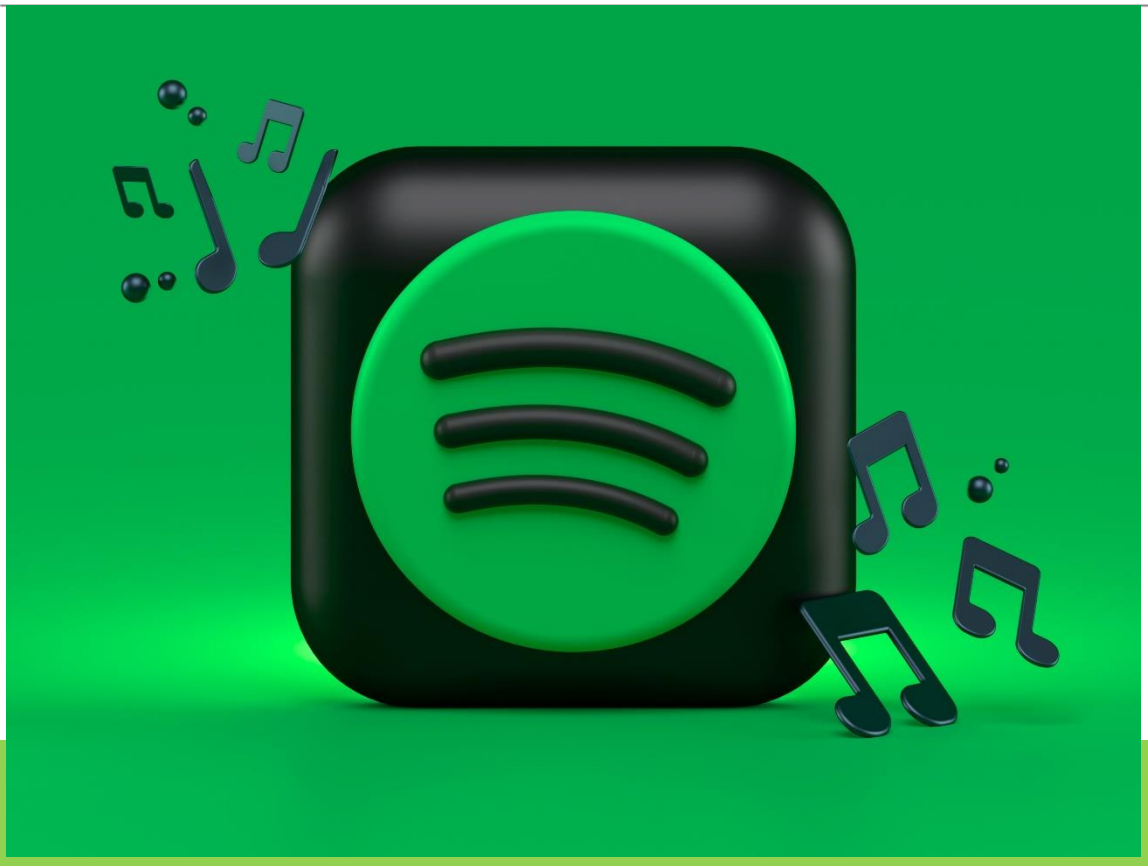
PROYECTO SPOTIFY



Mario GRACIA MARCO

La obtención de los datos

Este paso es el primero y es fundamental, porque para poder crear unas estadísticas, necesitamos los datos en sí. En mi caso yo he utilizado un playlist propia.

He creado un script para descargar las canciones de la playlist, no las canciones en sí, sino los metadatos que tienen.

```
import spotipy
from spotipy.oauth2 import SpotifyOAuth
import pandas as pd
from dotenv import load_dotenv
import os

load_dotenv()

# ======================
# 1. CONFIGURACIÓN DE AUTENTICACIÓN
# ======================
CLIENT_ID = os.getenv("CLIENT_ID")
CLIENT_SECRET = os.getenv("CLIENT_SECRET")
REDIRECT_URI = os.getenv("REDIRECT_URI")
SCOPE = "playlist-read-private"

# Crear objeto de autenticación
sp = spotipy.Spotify(auth_manager=SpotifyOAuth(
    client_id=CLIENT_ID,
    client_secret=CLIENT_SECRET,
    redirect_uri=REDIRECT_URI,
    scope=SCOPE
))
```

Primero hay que importar todas las librerías necesarias y cargar la función dotenv para no exponer datos sensibles.

Primero declaro mis credenciales:

- Id
- Client_secret
- La url donde se va a alojar la aplicación

Después las utilizo para conectar con Spotify.

```python
def fetch_playlist_tracks(playlist_id):
    """
    Devuelve todas las canciones de una playlist en una lista de diccionarios
    """
    results = sp.playlist_items(playlist_id, additional_types=["track"])
    tracks = results["items"]

    # Si hay más de 100 canciones, paginamos
    while results['next']:
        results = sp.next(results)
        tracks.extend(results['items'])

    all_tracks = []
    for item in tracks:
        track = item.get("track")
        if track:  # Verificamos que haya un track (para evitar errores)
            all_tracks.append({
                "track_name": track["name"],
                "artist": ", ".join([artist["name"] for artist in track["artists"]]),
                "album": track["album"]["name"],
                "release_date": track["album"]["release_date"],
                "popularity": track["popularity"],
                "duration_ms": track["duration_ms"],
                "track_url": track["external_urls"]["spotify"]
            })
    return all_tracks
```

Creo una función que crea un array donde se guardan los datos de las canciones, esto es como una base donde guardar los datos.

```python
playlist_id = "https://open.spotify.com/playlist/0HWA092vyoNiKlCip3rJ8j?si=ff0dde6a327d461e"

tracks_data = fetch_playlist_tracks(playlist_id)

# ====================
# 4. GUARDAR EN CSV
# ====================
df = pd.DataFrame(tracks_data)
df.to_csv("playlist.csv", index=False)
print(f"Se han descargado {len(df)} canciones y guardado en 'playlist.csv'")

# ====================
# 5. VISTA PREVIA
# ====================
print(df.head())
```

Después creo la variable que equivale a la url de la playlist y el script va cogiendo los datos de esa url y, a continuación, crea un csv y los va guardando ahí con la estructura de la función anterior.

```csv
track_name,artist,album,popularity,duration_ms
Soldadito marinero,Fito y Fitipaldis,Lo mas lejos a tu lado,69,254373
Lloverá y Yo Veré,La Pegatina,Eureka!,0,176718
Y Viva España,Manolo Escobar,"La Copla, Vol.4",1,216466
So payaso,Extremoduro,Agila,66,281666
Entre dos mares,Fito y Fitipaldis,Entre dos mares,55,280933
Pacto Entre Caballeros,Joaquín Sabina,Rock en Español,39,236160
A Contracorriente,El Canto Del Loco,Rock en Español,37,182346
Salir,Extremoduro,Canciones Prohibidas,65,318986
Por la Raja de Tu Falda,Estopa,Estopa,65,202826
El Toro Guapo,El Fary,Four Hits: El Fary,25,208520
15 d'Agosto,Ixo Rai,20 Grandes exitos,14,204786
Chiquilla,Seguridad Social,Que No Se Extinga La Llama!,58,187999
Feo Fuerte y Formal,Loquillo Y Los Trogloditas,"Feo, Fuerte y Formal",57,242266
```

Una vez ejecutado ese script queda este csv, con los datos del

- Nombre de la canción
- Artista
- Album
- Popularidad (lo determinan las métricas de Spotify)
- Duración (en milisegundos)

La utilización de los datos

Una vez tenemos los datos en el csv, hay que hacer otro script para mostrar los datos de una forma visual.

```python
import streamlit as st
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
import numpy as np

# ── CONFIGURACIÓN ──────────────────────────
st.set_page_config(
    page_title="Spotify Dashboard",
    page_icon="🎵",
    layout="wide",
    initial_sidebar_state="expanded",
)
```

Importo las librerías necesarias para que se puedan hacer los gráficos y las animaciones

Y determino la configuración básica que va a tener la app

```
st.markdown("""
<style>
@import url('https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700;800;900&di

/* ── Reset & Base ── */
html, body, [class*="css"] { font-family: 'Inter', sans-serif; }
.main { background: #050505; }
.block-container { padding: 1.5rem 2rem 3rem 2rem; background: #050505; }
section[data-testid="stSidebar"] { background: #0a0a0a !important; border-right: 1px solid #1a1

/* ── Animaciones ── */
@keyframes fadeSlideDown {
    from { opacity: 0; transform: translateY(-20px); }
    to   { opacity: 1; transform: translateY(0); }
}
```

Aquí están los css que va a tener la app, básicamente son líneas y líneas de código que determinan el formato que van a tener las gráficas, las letras y demás contenido.

```
@st.cache_data
def load_data():
    for fname in ["dataset.csv", "playlist.csv"]:
        try:
            return pd.read_csv(fname)
        except FileNotFoundError:
            continue
    st.error("❌ No se encontró 'dataset.csv' ni 'playlist.csv'. Ejecuta primero Spotify_fetch
    st.stop()

df_raw = load_data()
```

Cargo el dataset en la función de load_data y utilizo un try catch por si ocurre algún fallo a la hora de encontrar el dataset.

```
if 'duration_ms' in df_raw.columns:
    df_raw['duration_min'] = (df_raw['duration_ms'] / 60000).round(2)
```

Convierto los milisegundos que dura la canción a minutos.

```
# ── FILTROS (inline, sin sidebar) ──────────────────────────────────────
pop_min, pop_max = int(df_raw['popularity'].min()), int(df_raw['popularity'].max())
all_artists = sorted(df_raw['artist'].unique())
all_albums  = sorted(df_raw['album'].unique()) if 'album' in df_raw.columns else []

with st.expander("🎛️  Filtros  —  haz clic para mostrar/ocultar", expanded=False):
    fc1, fc2, fc3 = st.columns([2, 3, 3])
    with fc1:
        pop_range = st.slider("🔥 Popularidad", pop_min, pop_max, (pop_min, pop_max))
    with fc2:
        selected_artists = st.multiselect("🎤 Artistas", all_artists, placeholder="Todos los a
    with fc3:
        selected_albums = st.multiselect("💿 Álbumes", all_albums, placeholder="Todos los álbu
```

Esta es la configuración de los filtros, la base de los filtros que va a tener la barra.

```python
# ── APLICAR FILTROS ──────────────────────────────────────────────
df = df_raw.copy()
df = df[(df['popularity'] >= pop_range[0]) & (df['popularity'] <= pop_range[1])]
if selected_artists:
    df = df[df['artist'].isin(selected_artists)]
if selected_albums and 'album' in df.columns:
    df = df[df['album'].isin(selected_albums)]

if df.empty:
    st.warning("🔍 No hay canciones con los filtros aplicados.")
    st.stop()
```

Aquí se aplican los filtros.

```python
# ── MÉTRICAS ─────────────────────────────────────────────────────
total_songs     = len(df)
total_artists   = df['artist'].nunique()
total_albums    = df['album'].nunique() if 'album' in df.columns else 0
avg_pop         = round(df['popularity'].mean(), 1)
max_pop         = int(df['popularity'].max())
avg_dur_min     = round(df['duration_min'].mean(), 2) if 'duration_min' in df.columns else 0
total_hours     = round(df['duration_min'].sum() / 60, 1) if 'duration_min' in df.columns else
pct_popular     = round((df[df['popularity'] >= 70].shape[0] / total_songs) * 100, 1)
top_song        = df.loc[df['popularity'].idxmax(), 'track_name'] if 'track_name' in df.columns
top_artist_name = df['artist'].value_counts().idxmax()
top_artist_cnt  = df['artist'].value_counts().max()
rarest_album    = df['album'].value_counts()[-1:].index[0] if 'album' in df.columns and total_a
pop_std         = round(df['popularity'].std(), 1)
```

Saca todas las métricas que después se utilizaran para hacer los KPIs.

```python
# ── HEADER ───────────────────────────────────────────────────────
st.markdown(f"""
<div class="hero">
    <div class="hero-title">🎵 Spotify Dashboard</div>
    <div class="hero-sub">Análisis interactivo de tu playlist de música española</div>
    <div class="hero-badge">✦ {total_songs} canciones · {total_artists} artistas · {total_albu
</div>
""", unsafe_allow_html=True)
```

Esta es la cabecera de la página.

```
# Filtros activos
if selected_artists or selected_albums or pop_range != (pop_min, pop_max):
    pills = ""
    if pop_range != (pop_min, pop_max):
        pills += f'<span class="pill">🔥 Pop {pop_range[0]}-{pop_range[1]}</span>'
    for a in selected_artists:
        pills += f'<span class="pill">🎤 {a}</span>'
    for al in selected_albums:
        pills += f'<span class="pill">💿 {al}</span>'
    st.markdown(f'<div style="margin-bottom:1rem">{pills}</div>', unsafe_allow_html=True)
```

Estos son los filtros por defecto.

```
st.markdown('<div class="sec-header">📊 Indicadores Clave</div>', unsafe_allow_html=True)
c1, c2, c3, c4 = st.columns(4)

def kpi(col, emoji, label, value, unit="", sub="", color="#1DB954", bar_pct=0):
    bar_w = f"{min(bar_pct, 100):.0f}%"
    bar_html = f"""<div class="kpi-bar-bg"><div class="kpi-bar" style="--w:{bar_w};--c:{color}'
    small_cls = "small" if len(str(value)) > 12 else ""
    with col:
        st.markdown(f"""
        <div class="kpi-wrap">
          <div class="kpi-card" style="--c:{color}">
            <div class="kpi-glow"></div>
            <div class="kpi-emoji">{emoji}</div>
            <div class="kpi-label">{label}</div>
            <div class="kpi-value {small_cls}">{value}<span class="kpi-unit">{unit}</span></di
            <div class="kpi-sub">{sub}</div>
            {bar_html}
          </div>
        </div>""", unsafe_allow_html=True)
```

Aquí se aplica el formato a los KPIs.

```
kpi(c1, "🎵", "Total Canciones",    total_songs,      sub=f"{total_albums} álbumes distintos",
kpi(c2, "🎤", "Artistas Únicos",    total_artists,    sub=f"Top: {top_artist_name[:20]}",
kpi(c3, "🔥", "Popularidad Prom.",  avg_pop, unit=" pts", sub=f"Máxima: {max_pop} · Desv: {pop_
kpi(c4, "⏱", "Duración Promedio",  avg_dur_min, unit=" min", sub=f"Total: {total_hours}h de m

c5, c6, c7, c8 = st.columns(4)
top_name_sh top_artist_cnt = df['artist'].value_counts().max()  top_song
kpi(c5, "★                                                      e: {max_pop} / 100",
kpi(c6, "🏆 Full name: app.top_artist_cnt                       " if len(top_artist_name)>20 el
    sub=f"{top_artist_cnt} canciones en la lista",
kpi(c7, "🚀", "Canciones Virales",   f"{pct_popular}%", sub="Popularidad ≥ 70",
kpi(c8, "💿", "Álbumes Distintos",   total_albums,   sub=f"Promedio {total_songs//max(total_al
```

Después utilizamos la función KPI para crear los distintos KPIs con los datos que queremos sacar. El total, los artistas, la popularidad…

Resultado de estos KPIs

```
# ── GRÁFICOS ──────────────────────────────────────────────────────────
PLOTLY_LAYOUT = dict(
    paper_bgcolor="#0d0d12",
    plot_bgcolor="#0d0d12",
    font=dict(family="Inter", color="#9ca3af", size=12),
    margin=dict(l=10, r=10, t=40, b=10),
    hoverlabel=dict(bgcolor="#1a1a2e", font_color="white", font_size=13, bordercolor="#333"),
    xaxis=dict(gridcolor="#1a1a2e", zeroline=False),
    yaxis=dict(gridcolor="#1a1a2e", zeroline=False),
)
```

Determino el formato de los gráficos.

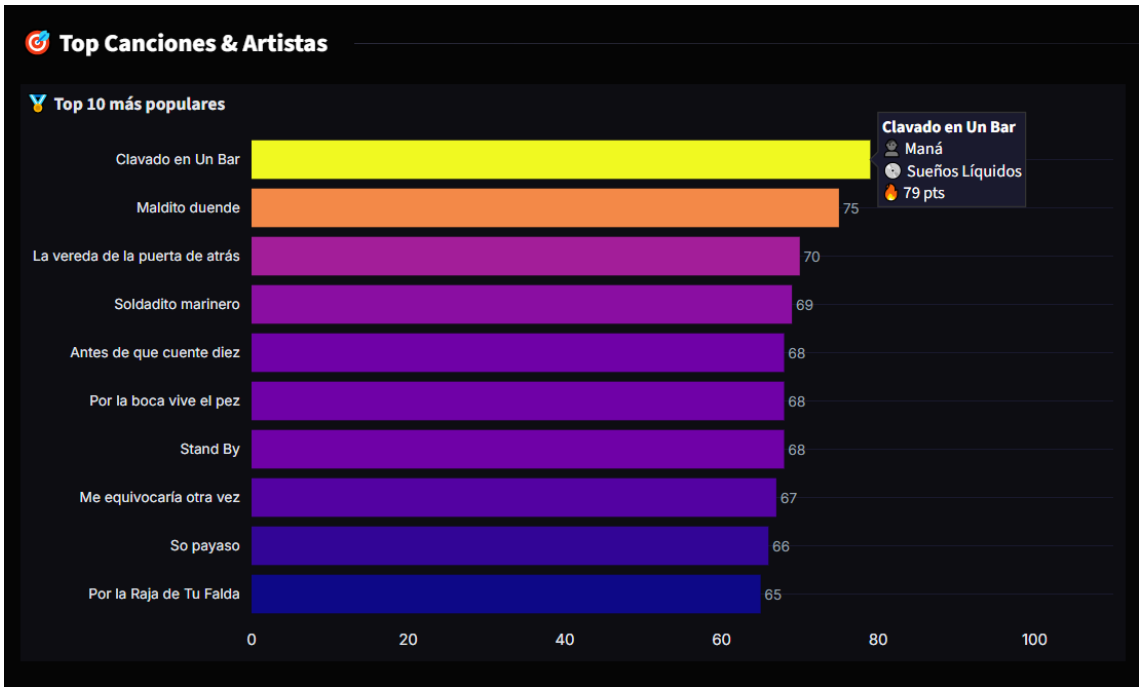
```
# ── Fila de gráficos – Top canciones + Donut Artistas ──────────────────
st.markdown('<div class="sec-header">🎯 Top Canciones & Artistas</div>', unsafe_allow_html=True)
col_l, col_r = st.columns([3, 2], gap="medium")

with col_l:
    top10 = df.sort_values('popularity', ascending=False).head(10).copy()
    top10['label'] = top10['track_name'].str[:32]

    palette = px.colors.sequential.Plasma_r[:10]
    fig = go.Figure(go.Bar(
        x=top10['popularity'],
        y=top10['label'],
        orientation='h',
        marker=dict(
            color=top10['popularity'],
            colorscale='Plasma',
            line=dict(width=0),
        ),
        text=top10['popularity'],
        textposition='outside',
        textfont=dict(size=11, color='#9ca3af'),
        customdata=top10[['artist', 'album']],
        hovertemplate="<b>%{y}</b><br>🎤 %{customdata[0]}<br>💿 %{customdata[1]}<br>🔥 %{x} pts<extra></extra>",
    ))
    fig.update_layout(**PLOTLY_LAYOUT, title="🏆 Top 10 más populares",
                      title_font=dict(size=14, color="#e5e7eb"),
                      xaxis_range=[0, 110])
    fig.update_yaxes(autorange="reversed", tickfont=dict(size=11))
    fig.update_traces(marker_line_width=0)
    st.plotly_chart(fig, use_container_width=True, config={"displayModeBar": False})
```

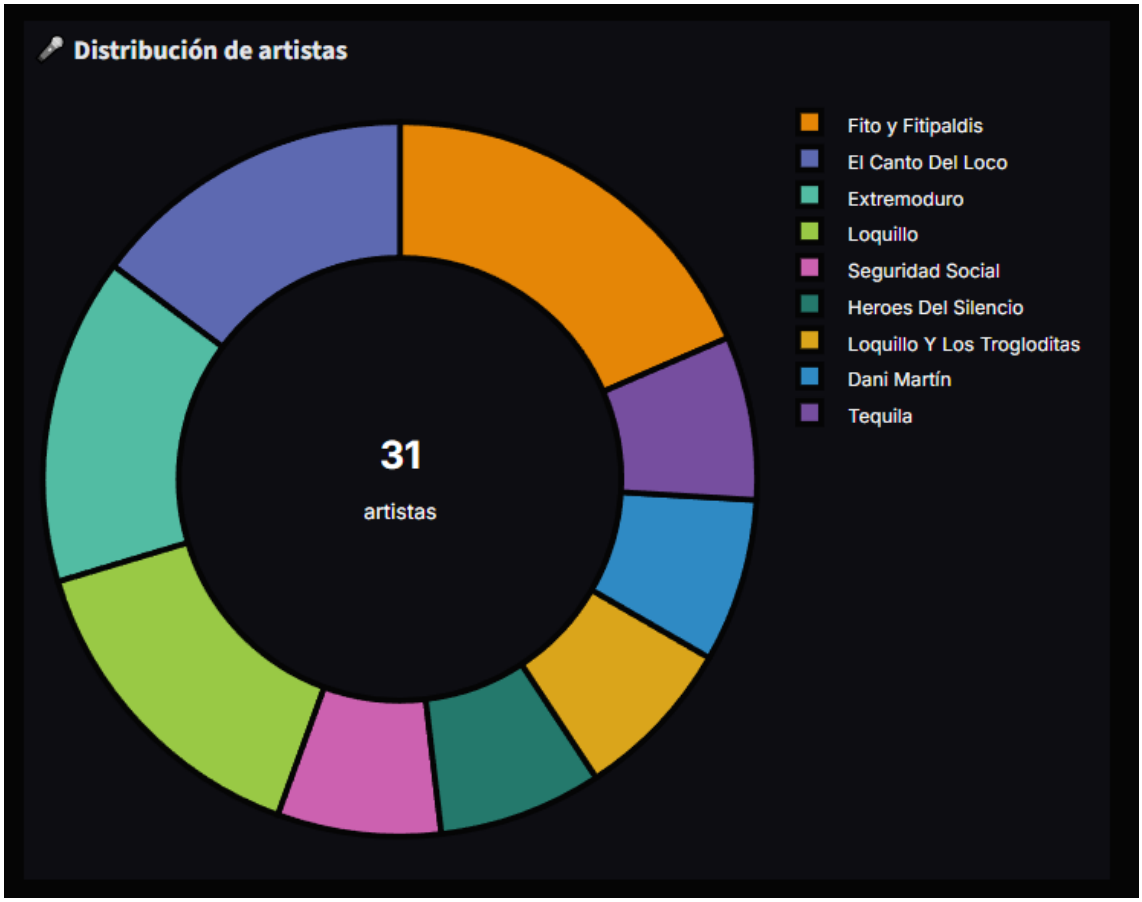
Contenido del primer gráfico.

El dato principal es la popularidad, pero al poner el cursor encima de cada barra muestra más información.




```python
with col_r:
    artist_counts = df['artist'].value_counts().head(9)
    fig2 = go.Figure(go.Pie(
        labels=artist_counts.index,
        values=artist_counts.values,
        hole=0.62,
        marker=dict(
            colors=px.colors.qualitative.Vivid,
            line=dict(color='#050505', width=3)
        ),
        textinfo='none',
        hovertemplate="<b>%{label}</b><br>%{value} canciones · %{percent}<extra></extra>",
    ))
    fig2.add_annotation(
        text=f"<b>{total_artists}</b><br><span style='font-size:11px'>artistas</span>",
        x=0.5, y=0.5, showarrow=False,
        font=dict(size=20, color='white'),
    )
    fig2.update_layout(**PLOTLY_LAYOUT, title="🎤 Distribución de artistas",
                       title_font=dict(size=14, color="#e5e7eb"),
                       legend=dict(font=dict(size=10), bgcolor="rgba(0,0,0,0)"))
    st.plotly_chart(fig2, use_container_width=True, config={"displayModeBar": False})
```

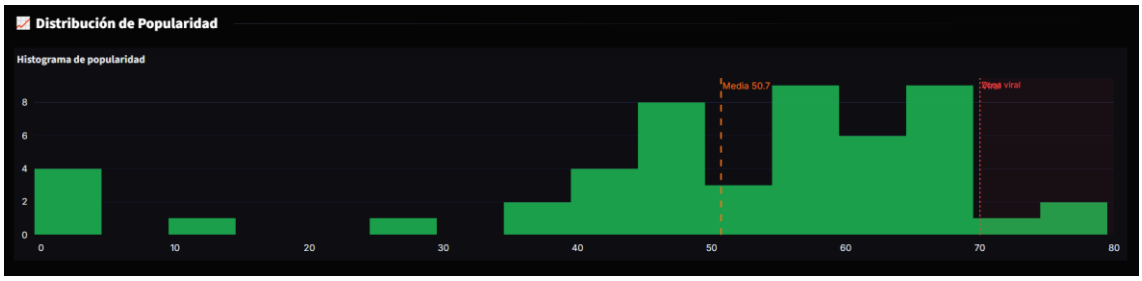
Este es el quesito con los artistas.




```python
# ── Distribución de popularidad ──────────────────────────────────────────
st.markdown('<div class="sec-header">📈 Distribución de Popularidad</div>', unsafe_allow_html=True)

fig3 = go.Figure()
fig3.add_trace(go.Histogram(
    x=df['popularity'], nbinsx=25,
    marker=dict(
        color='#1DB954',
        line=dict(width=0),
    ),
    name="Canciones",
    hovertemplate="Popularidad %{x}<br>%{y} canciones<extra></extra>",
    opacity=0.85,
))
fig3.add_vline(x=avg_pop, line_dash="dash", line_color="#f97316", line_width=2,
               annotation_text=f"Media {avg_pop}", annotation_font_color="#f97316")
fig3.add_vline(x=70, line_dash="dot", line_color="#ef4444", line_width=1.5,
               annotation_text="Viral", annotation_font_color="#ef4444")
fig3.add_vrect(x0=70, x1=df['popularity'].max()+1,
               fillcolor="rgba(239,68,68,0.05)", line_width=0,
               annotation_text="Zona viral", annotation_position="top left",
               annotation_font_color="#ef4444", annotation_font_size=11)
fig3.update_layout(**PLOTLY_LAYOUT, height=280,
                   title="Histograma de popularidad", title_font=dict(size=14, color="#e5e7eb"))
st.plotly_chart(fig3, use_container_width=True, config={"displayModeBar": False})
```

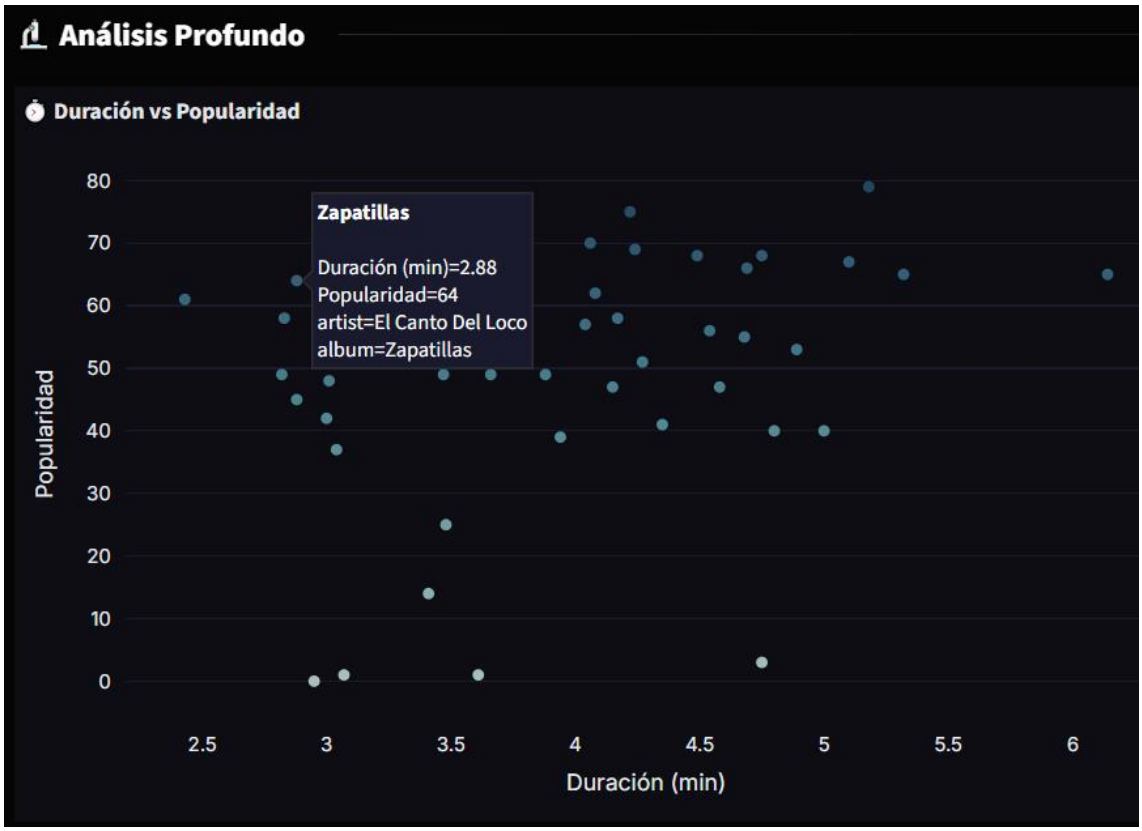
Este gráfico da información con el porcentaje de popularidad de las canciones y el número de canciones que tienen ese porcentaje de popularidad.




```python
# ── Fila – Scatter + Top Artistas barras ──────────────────────────────
st.markdown('<div class="sec-header">🔬 Análisis Profundo</div>', unsafe_allow_html=True)
col3, col4 = st.columns(2, gap="medium")

with col3:
    fig4 = px.scatter(
        df, x='duration_min', y='popularity',
        color='popularity',
        color_continuous_scale='Teal',
        hover_name='track_name' if 'track_name' in df.columns else None,
        hover_data={'artist': True, 'album': True, 'duration_min': ':.2f', 'popularity': True},
        labels={'duration_min': 'Duración (min)', 'popularity': 'Popularidad'},
        title="⏱ Duración vs Popularidad",
        size_max=10,
    )
    fig4.update_traces(marker=dict(size=7, opacity=0.8, line=dict(width=0)))
    fig4.update_layout(**PLOTLY_LAYOUT, coloraxis_showscale=False,
                       title_font=dict(size=14, color="#e5e7eb"))
    st.plotly_chart(fig4, use_container_width=True, config={"displayModeBar": False})
```

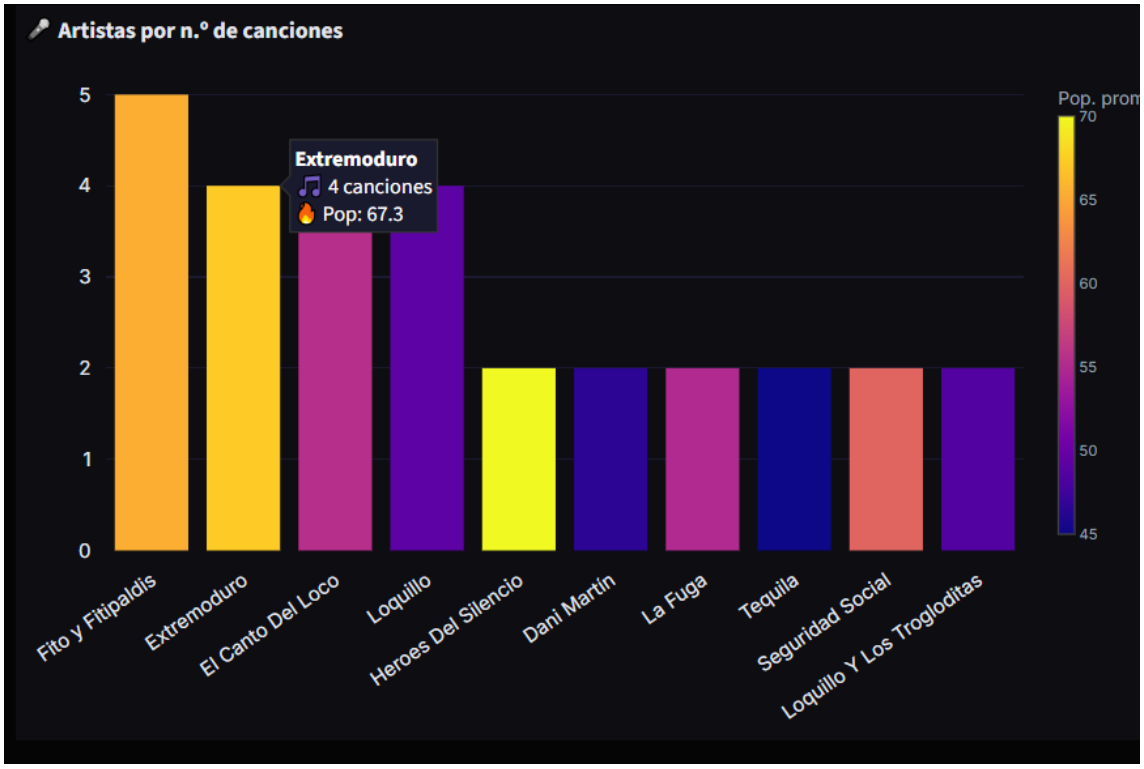
Esta función analiza la popularidad de la canción con su duración. En este caso, son pocas canciones, pero a escala más grande puede buscar una correlación entre la popularidad y duración.




```python
with col4:
    top_artists_df = df.groupby('artist').agg(
        canciones=('track_name', 'count'),
        pop_promedio=('popularity', 'mean')
    ).sort_values('canciones', ascending=False).head(10).reset_index()

    fig5 = go.Figure()
    fig5.add_trace(go.Bar(
        x=top_artists_df['artist'],
        y=top_artists_df['canciones'],
        name='Canciones',
        marker=dict(
            color=top_artists_df['pop_promedio'],
            colorscale='Plasma',
            showscale=True,
            colorbar=dict(title="Pop. prom.", tickfont=dict(size=9), thickness=10),
        ),
        hovertemplate="<b>%{x}</b><br>🎵 %{y} canciones<br>🔥 Pop: %{marker.color:.1f}<extra></extra>",
    ))
    fig5.update_layout(**PLOTLY_LAYOUT, title="🎤 Artistas por n.º de canciones",
                       title_font=dict(size=14, color="#e5e7eb"),
                       xaxis_tickangle=-35)
    st.plotly_chart(fig5, use_container_width=True, config={"displayModeBar": False})
```

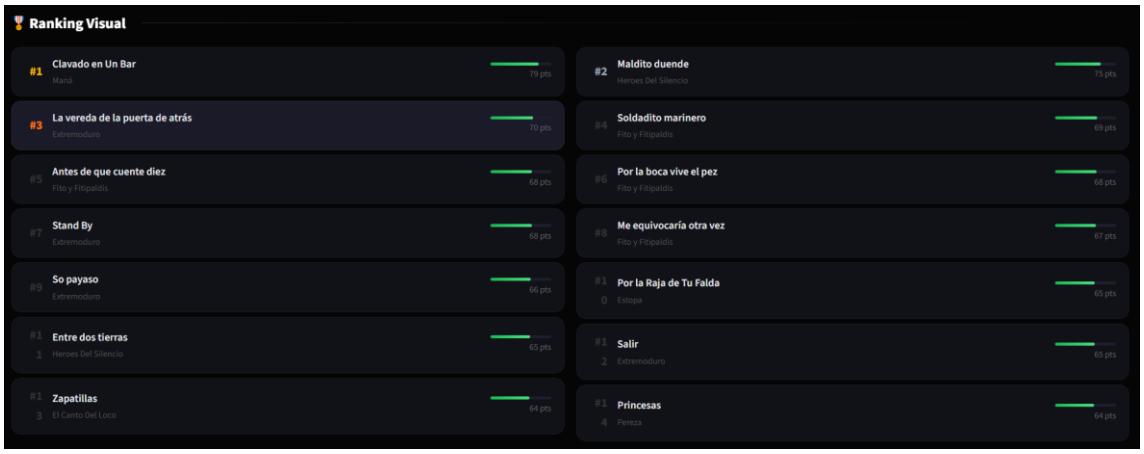
Esta es la gráfica que está al lado de la anterior y muestra el numero de canciones por artista.




```python
# ── TOP 10 lista visual ──────────────────────────────────────────────
st.markdown('<div class="sec-header">🏆 Ranking Visual</div>', unsafe_allow_html=True)
cols_rank = st.columns(2)
top15 = df.sort_values('popularity', ascending=False).head(14)

for idx, (_, row) in enumerate(top15.iterrows()):
    col_idx = idx % 2
    rank = idx + 1
    rank_cls = "gold" if rank == 1 else "silver" if rank == 2 else "bronze" if rank == 3 else ""
    name = row.get('track_name', 'N/A')[:40]
    artist = row.get('artist', 'N/A')[:30]
    pop = int(row['popularity'])
    pop_pct = f"{pop}%"
    delay = f"{0.05 * idx:.2f}s"
    with cols_rank[col_idx]:
        st.markdown(f"""
        <div class="song-row" style="animation-delay:{delay}">
            <div class="song-rank {rank_cls}">#{rank}</div>
            <div class="song-info">
                <div class="song-name">{name}</div>
                <div class="song-artist">{artist}</div>
            </div>
            <div class="song-pop-bar">
                <div class="pop-bg"><div class="pop-fill" style="width:{pop_pct}"></div></div>
                <div class="song-pop-num">{pop} pts</div>
            </div>
        </div>""", unsafe_allow_html=True)
```

Esta función hace un ranking por popularidad.




```python
# ── TABLA INTERACTIVA ──────────────────────────────────────────
st.markdown('<div class="sec-header">📁 Explorador de Canciones</div>', unsafe_allow_html=True)

search = st.text_input("🔍 Buscar canción o artista...", placeholder="Escribe para filtrar...")
df_show = df.copy()
if search:
    mask = (
        df_show['track_name'].str.contains(search, case=False, na=False) |
        df_show['artist'].str.contains(search, case=False, na=False)
    )
    df_show = df_show[mask]

display_cols = [c for c in ['track_name', 'artist', 'album', 'popularity', 'duration_min'] if c in df_show.columns]
df_display = df_show[display_cols].copy()

st.dataframe(
    df_display,
    use_container_width=True,
    height=420,
    column_config={
        "track_name":   st.column_config.TextColumn("🎵 Canción"),
        "artist":       st.column_config.TextColumn("🎤 Artista"),
        "album":        st.column_config.TextColumn("💿 Álbum"),
        "popularity":   st.column_config.ProgressColumn("🔥 Popularidad", min_value=0, max_value=100, format="%d"),
        "duration_min": st.column_config.NumberColumn("⏱ Min", format="%.2f"),
    },
    hide_index=True,
)

st.markdown(f"<p style='color:#333;font-size:0.72rem;text-align:right'>Mostrando {len(df_show)} de {total_songs} canciones</p>", uns
```

Y, por último, una tabla en la que aparecen todos los datos. Aquí no hay ningún orden específico, por eso tiene una barra de búsqueda para filtrar por x característica.

📁 Explorador de Canciones

🔍 Buscar canción o artista...

Escribe para filtrar...

🎵 Canción	🎤 Artista	💿 Álbum	🔥 Popularidad	⏱ Min
Soldadito marinero	Fito y Fitipaldis	Lo mas lejos a tu lado	69	4.24
Lloverá y Yo Veré	La Pegatina	Eureka!	0	2.95
Y Viva España	Manolo Escobar	La Copla, Vol.4	1	3.61
So payaso	Extremoduro	Agila	66	4.69
Entre dos mares	Fito y Fitipaldis	Entre dos mares	55	4.68
Pacto Entre Caballeros	Joaquín Sabina	Rock en Español	39	3.94
A Contracorriente	El Canto Del Loco	Rock en Español	37	3.04
Salir	Extremoduro	Canciones Prohibidas	65	5.32
Por la Raja de Tu Falda	Estopa	Estopa	65	3.38
El Toro Guapo	El Fary	Four Hits: El Fary	25	3.48
15 d'Agosto	Ixo Rai	20 Grandes exitos	14	3.41

La importancia del dato

Como conclusión final, el dato es algo muy importante. No hace falta hacer un trabajo así para darse cuenta, ya que solo con ver la cantidad de aplicaciones “gratis” que existen, es una prueba más que suficiente.

En relación con el trabajo, te das cuenta de como las grandes compañías, especialmente las tecnológicas utilizan nuestros datos para sus propios intereses comerciales. Los datos al final dan contexto a las situaciones.

Yo he utilizado una playlist de música española, estilo pop-rock, aunque también con más estilos. Esto es interesante para Spotify para saber que estilo musical me gusta y que recomendaciones me puede hacer. En lo personal es una idea muy interesante ya que tú como cliente tienes la oportunidad de descubrir música que no conocías y ha resultado que te gusta o otras canciones que conocías, pero no recordabas el nombre.

En definitiva, nada es gratis, los datos valen muchísimo para las empresas y nosotros como individuos debemos tener control sobre ellos y no regalarlos por cualquier cosa.

Link del repositorio

https://github.com/gracia25-star/ProyectoSpotify.git